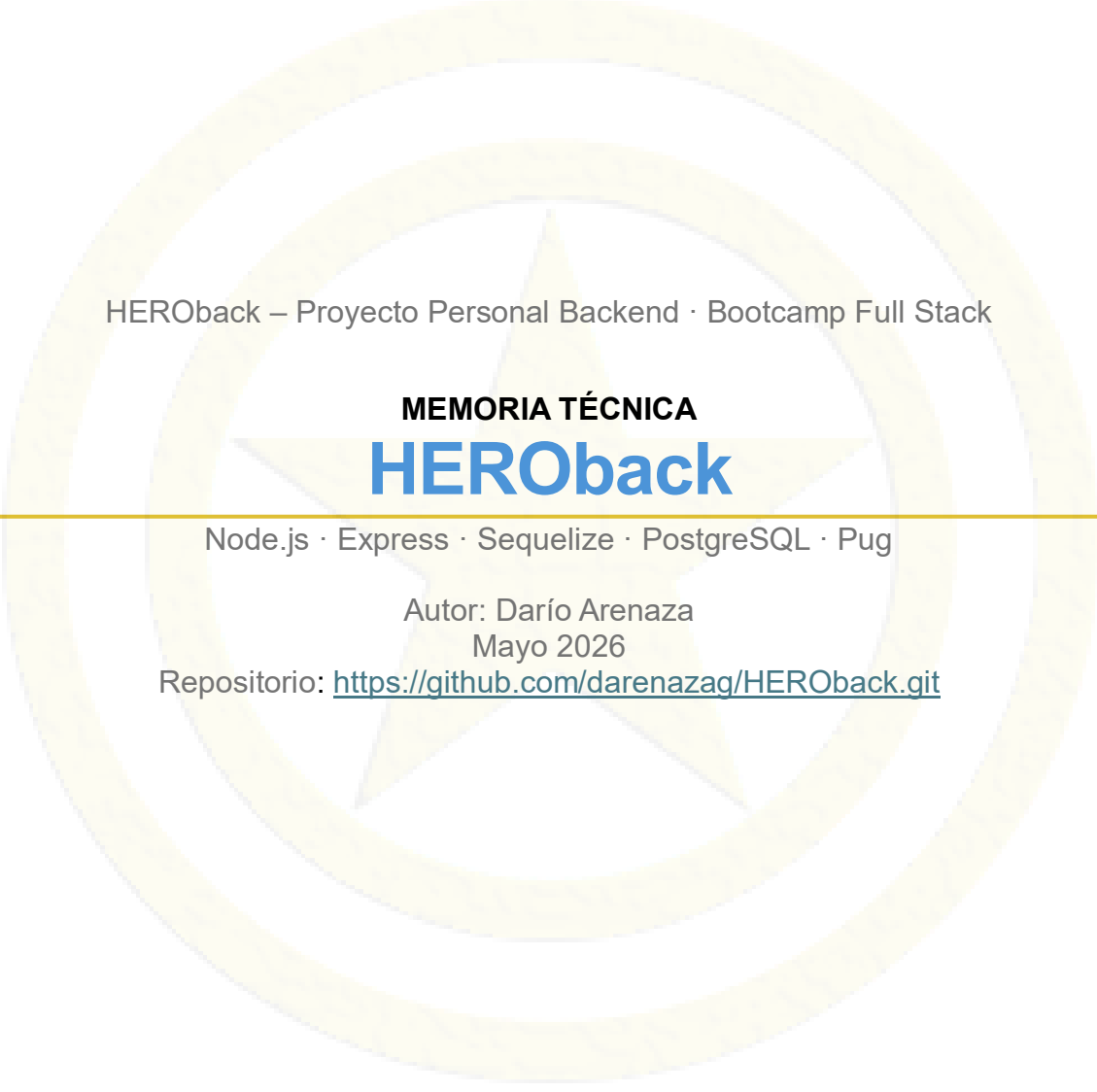




HEROback – Proyecto Personal Backend · Bootcamp Full Stack

**MEMORIA TÉCNICA**  
**HEROback**

---

Node.js · Express · Sequelize · PostgreSQL · Pug

Autor: Darío Arenaza  
Mayo 2026

Repositorio: <https://github.com/darenazag/HEROback.git>

# Índice

---

## Contenido

|                                                |    |
|------------------------------------------------|----|
| 1. Descripción General.....                    | 2  |
| 2. Análisis de Requisitos .....                | 2  |
| 3. Objetivos .....                             | 3  |
| 4. Stack Tecnológico y Justificación .....     | 3  |
| 5. Arquitectura del Proyecto .....             | 4  |
| 5.1. Middlewares clave .....                   | 5  |
| 5.2. Estructura de carpetas .....              | 5  |
| 6. Modelos de Datos .....                      | 6  |
| 6.1. Asociaciones Sequelize .....              | 7  |
| 6.2. Esquema de relaciones .....               | 7  |
| 6.3. Decisiones clave .....                    | 7  |
| 7. API RESTful.....                            | 8  |
| 7.1. Paginación, filtrado y ordenamiento ..... | 8  |
| 8. Seguridad y Autenticación .....             | 9  |
| 9. Seeding de datos.....                       | 10 |
| 10. Despliegue .....                           | 10 |
| 11. Lógica de negocio destacada.....           | 10 |
| 12. Retos encontrados y soluciones .....       | 11 |
| 13. Conclusiones .....                         | 11 |
| 14. Mejoras futuras.....                       | 11 |

## 1. Descripción General

---

HEROback es el backend de una plataforma de superhéroes que permite a los usuarios registrarse, explorar un catálogo importado de la SuperHero API, coleccionar héroes, comprar productos en una tienda interna y formar equipos de combate. La aplicación expone tanto vistas server-side (Pug) como una API REST independiente documentada con OpenAPI.

## 2. Análisis de Requisitos

---

| Requisitos                           | Implementación                                             |
|--------------------------------------|------------------------------------------------------------|
| Repositorio + README                 | README.md exhaustivo                                       |
| Memoria del proyecto                 | Este documento                                             |
| >=3 tablas relacionadas              | 7 tablas con FK                                            |
| Registro y login funcional           | POST /api/auth/register, /login, vistas /register y /login |
| Roles / permisos                     | ENUM user_role + checkRole + requireAdmin                  |
| >=3 endpoints (>=2 que modifican BD) | Ver tabla sección 6                                        |
| Vistas con motor de plantillas Pug   | 12 vistas en src/views/*.pug                               |
| JSDoc                                | Aplicado en services, controllers, middlewares y rutas     |
| API RESTful independiente            | /api/* separado de vistas; mismo backend                   |
| ORM Sequelize                        | Sí, con asociaciones complejas                             |
| Paginación                           | ?page&limit (con tope a 100) en todos los listados         |
| Filtrado                             | ?campo__op=valor con whitelist de campos seguros           |
| Ordenamiento                         | ?sort=campo / ?sort=-campo,otro                            |
| Documentación de API                 | OpenAPI 3.0 disponible en /api/docs                        |

### 3. Objetivos

---

- Diseñar e implementar un backend funcional y mantenible.
- Persistir datos en PostgreSQL mediante Sequelize ORM.
- Exponer una API RESTful con paginación, filtrado y ordenamiento.
- Implementar autenticación dual (sesión para vistas, JWT para API).
- Documentar el código (JSDoc) y la API (Swagger / OpenAPI 3.0).

### 4. Stack Tecnológico y Justificación

---

| Capa           | Tecnología                 | Por qué                                                         |
|----------------|----------------------------|-----------------------------------------------------------------|
| Runtime        | Node.js 18+ (ES Modules)   | Sintaxis moderna import/export y top-level await.               |
| Framework HTTP | Express 4                  | Estable, minimalista, ecosistema enorme.                        |
| ORM            | Sequelize 6                | Soporta Postgres + asociaciones complejas.                      |
| Base de datos  | PostgreSQL 16              | Tipos ricos (ENUM, TIMESTAMPTZ), integridad referencial fuerte. |
| Vistas         | Pug                        | Sintaxis indentada limpia.                                      |
| Autenticación  | JWT + express-session      | JWT para API stateless, sesión para las vistas.                 |
| Cifrado        | bcryptjs (cost 12)         | Estándar para almacenar contraseñas.                            |
| Documentación  | swagger-jsdoc + swagger-ui | Spec generado a partir de comentarios JSDoc.                    |
| Contenedores   | docker-compose             | Entorno reproducible en cualquier máquina.                      |

## 5. Arquitectura del Proyecto

HEROback sigue una arquitectura por **capas** con separación de responsabilidades. Cada petición entra por Express, atraviesa los middlewares (sesión, autenticación, helpers de query), se enruta hacia controladores de API o de vistas, los cuales delegan la lógica en **servicios** que encapsulan el acceso a datos vía Sequelize.

Patrón clásico **routes** → **controllers** → **services** → **models**:

- **routes/** — define los endpoints, aplica middlewares y delega.
- **controllers/** — se ocupa de req y res. No contiene reglas de negocio.
- **services/** — lógica pura, fácil de testear; única capa que toca la BD.
- **models/** — esquemas Sequelize y asociaciones.

Esta separación permite reusar el mismo **productService.getAll** desde la API REST y desde el render de la vista /store.

Flujo de funcionamiento · HEROback

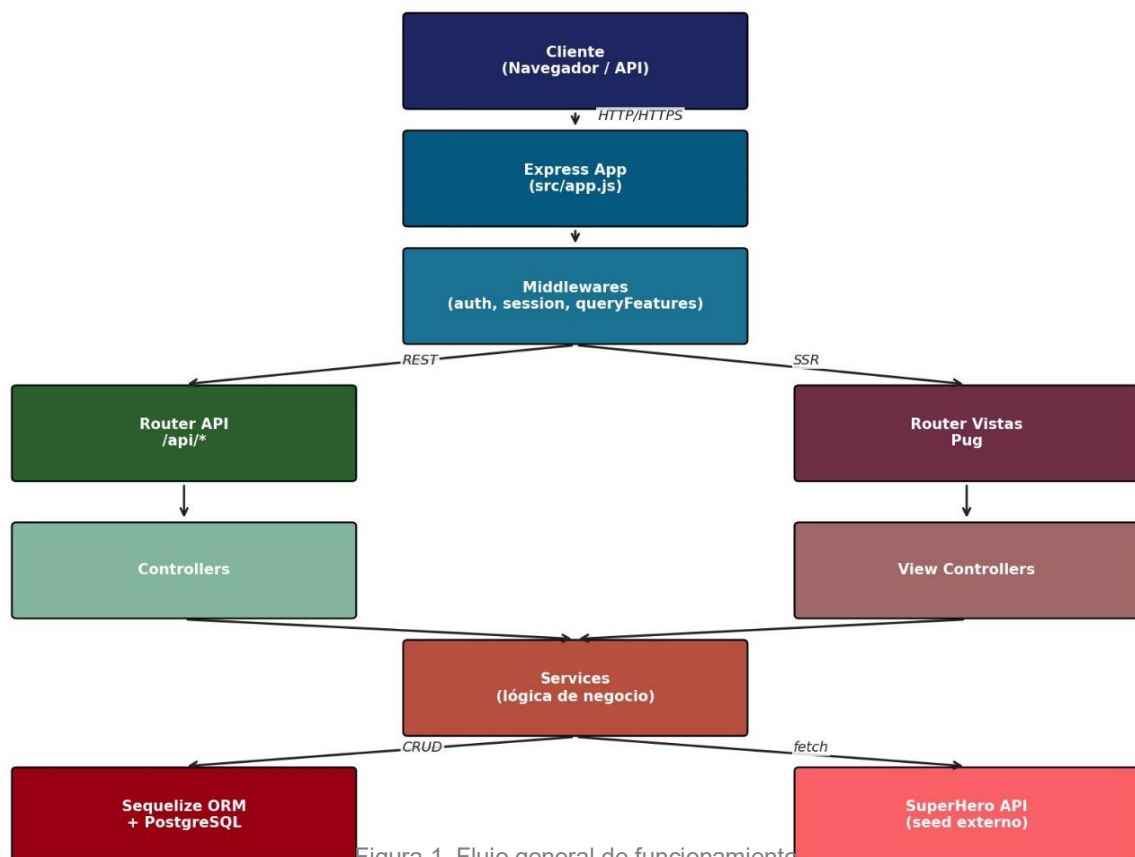


Figura 1. Flujo general de funcionamiento

## 5.1. Middlewares clave

- **authMiddleware.verifyToken + checkRole**: protección JWT para la API.
- **sessionMiddleware.requireSession + requireAdmin**: protección para las vistas Pug.
- **queryFeatures(filters, sorts)**: factory reusable que parsea `?page&limit;&sort;&campo;__op=valor`.  
Solo permite filtrar/ordenar campos whitelisted, protegiéndose contra inyección por querystring.

## 5.2. Estructura de carpetas

```
HEROback/
+-- src/
|   +-- app.js           # Configuración Express
|   +-- server.js        # Bootstrap + sync DB
|   +-- config/          # database.js, swagger.js
|   +-- models/          # Modelos Sequelize + asociaciones
|   +-- controllers/     # API y vistas
|   +-- services/        # Lógica de negocio
|   +-- routes/          # API + viewRoutes
|   +-- middlewares/     # auth, queryFeatures, errorHandler
|   +-- views/           # Plantillas Pug
|   +-- seeders/         # heroes.json (offline)
|   `-- public/css/      # styles.css mobile-first
+-- public/              # estáticos servidos en raíz
+-- scripts/seed.js      # ETL desde SuperHero API
+-- docker-compose.yml
`-- docs/MEMORIA.md
```

## 6. Modelos de Datos

El esquema relacional contempla siete entidades principales. Los usuarios crean equipos (teams) que agrupan héroes mediante una tabla intermedia (team\_heroes). Cada héroe puede tener múltiples poderes a través de hero\_powers. Los comentarios vinculan usuarios con héroes.

Diagrama Entidad-Relación · HEROback

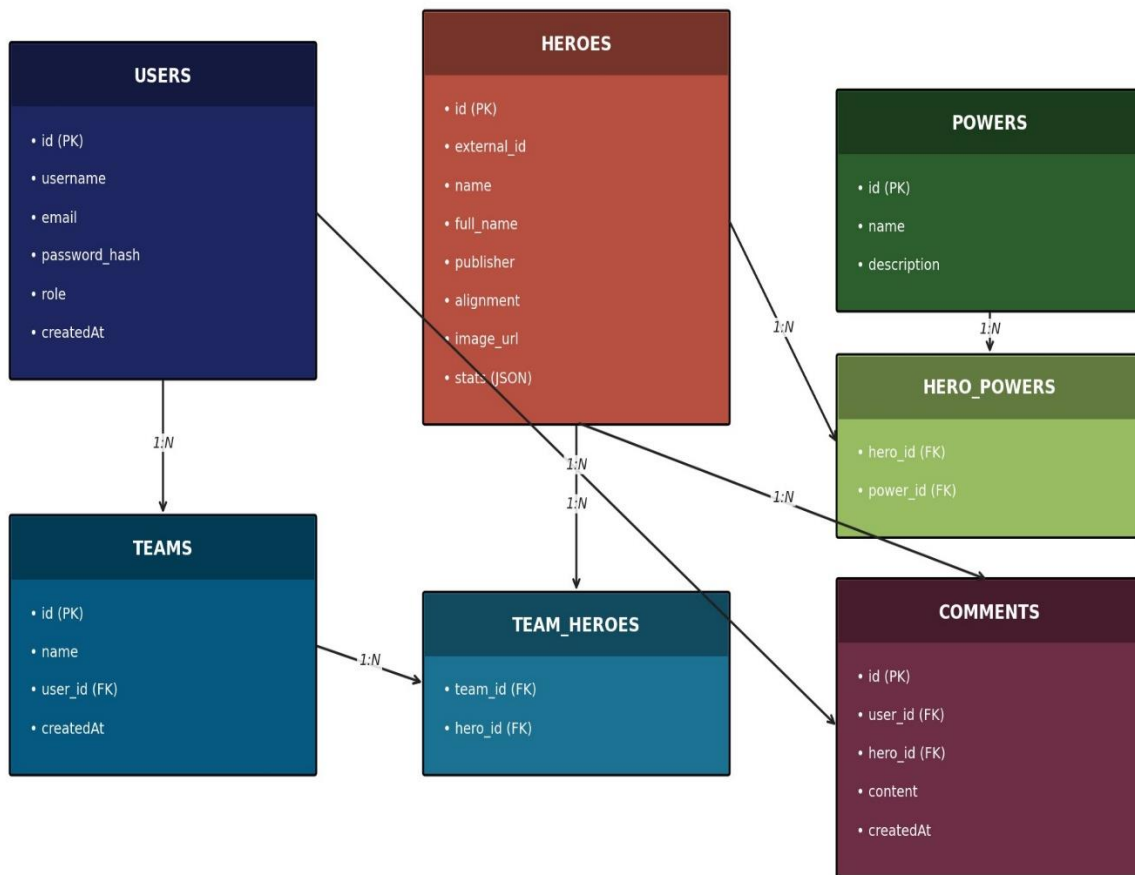


Figura 2. Diagrama Entidad-Relación.

## 6.1. Asociaciones Sequelize

```
User.hasMany(Team, { foreignKey: 'user_id' });
User.hasMany(Comment, { foreignKey: 'user_id' });
Team.belongsTo(User);
Team.belongsToMany(Hero, { through: TeamHero });
Hero.belongsToMany(Team, { through: TeamHero });
Hero.belongsToMany(Power, { through: HeroPower });
Power.belongsToMany(Hero, { through: HeroPower });
Hero.hasMany(Comment);
Comment.belongsTo(User);
Comment.belongsTo(Hero);
```

## 6.2. Esquema de relaciones

| Relación                  | Tipo        |
|---------------------------|-------------|
| users → user_heroes       | One-to-Many |
| heroes → user_heroes      | One-to-Many |
| users → user_products     | One-to-Many |
| products → user_products  | One-to-Many |
| users → teams             | One-to-Many |
| teams → team_heroes       | One-to-Many |
| user_heroes → team_heroes | Many-to-One |

## 6.3. Decisiones clave

- **Roles vía ENUM ('admin' | 'user')** en la tabla users: simple, suficiente para los requisitos.
- **user\_heroes con UNIQUE(user\_id, hero\_id)**: un mismo héroe no puede añadirse dos veces a la colección.
- **team\_heroes referencia a user\_heroes** (no a heroes) para garantizar que un equipo solo contenga héroes que el usuario posea.
- **Campos coins y stock con CHECK >= 0**: evita valores negativos ilegales aunque la lógica de negocio falle.



## 7. API RESTful

La API expone recursos siguiendo el estándar REST. Toda la documentación interactiva está disponible en **/api/docs** (Swagger UI).

| Método | Ruta                  | Descripción               |
|--------|-----------------------|---------------------------|
| POST   | /api/auth/register    | Registro de usuario       |
| POST   | /api/auth/login       | Login (devuelve JWT)      |
| GET    | /api/heroes           | Listado paginado/filtrado |
| GET    | /api/heroes/:id       | Detalle de héroe          |
| POST   | /api/teams            | Crear equipo (auth)       |
| GET    | /api/teams            | Equipos del usuario       |
| POST   | /api/teams/:id/heroes | Añadir héroe a equipo     |
| DELETE | /api/teams/:id        | Eliminar equipo           |
| GET    | /api/comments?herold= | Comentarios de un héroe   |
| POST   | /api/comments         | Crear comentario (auth)   |

### 7.1. Paginación, filtrado y ordenamiento

El middleware **queryFeatures** traduce parámetros de query a opciones Sequelize. Soporta operadores:

**`__like`, `__gte`, `__lte`, `__ne`, `__in`.**

```
GET /api/heroes
  ?page=2&limit=10
  &sort=-name           (descendente por nombre)
  &publisher=Marvel%20Comics
  &name__like=spider
  &alignment__ne=bad
```

## 8. Seguridad y Autenticación

HEROback implementa un esquema dual: **sesiones** con cookies firmadas para las vistas Pug y **JWT** (Bearer Token) para los consumidores de la API. Las contraseñas se almacenan con bcrypt (cost 12). Se incluye protección CSRF en formularios y validación estricta con express-validator.

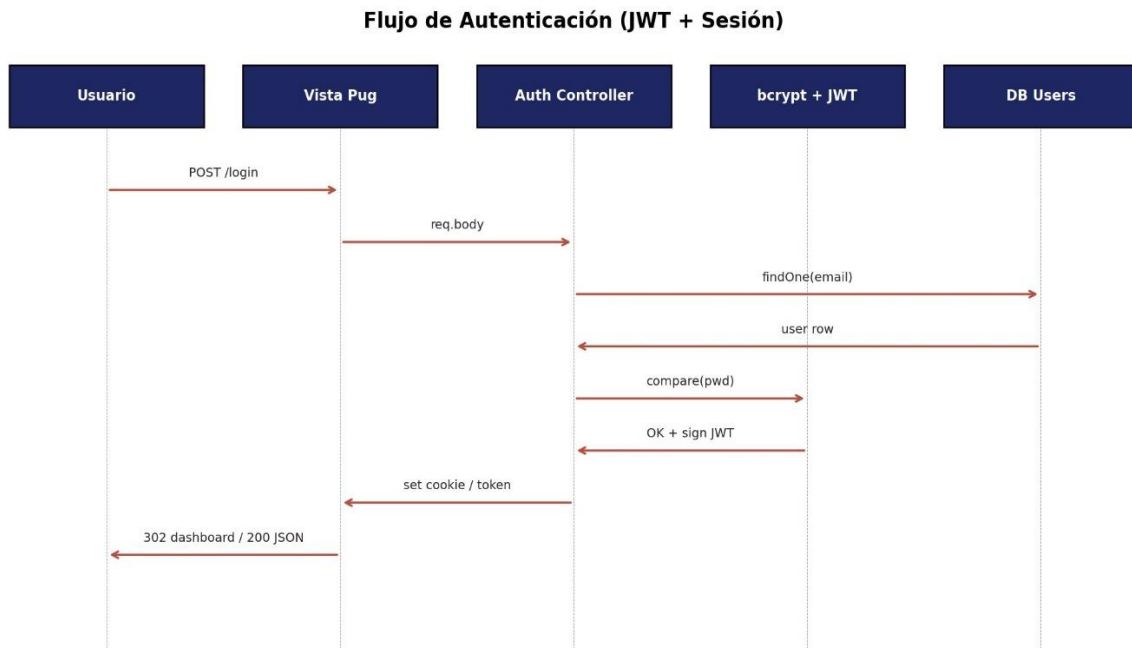


Figura 3. Flujo de autenticación.

- **Contraseñas:** bcrypt (cost 12), nunca expuestas en respuestas ni logs (password\_hash excluido vía attributes: exclude).
- **JWT:** Firmado con JWT\_SECRET, expira a los 7 días, transportado en Authorization: Bearer.
- **Sesiones:** Cookie httpOnly, secure en producción.
- **Autorización:** checkRole('admin') en routes para todas las mutaciones administrativas; usuarios solo pueden modificar su propio recurso.
- **Validación:** Checks manuales en controllers + validate de Sequelize (longitud mínima, isEmail, rangos min/max).
- **Transacciones:** userStoreService.buyProduct envuelve la compra en sequelize.transaction() con lock: true para evitar race conditions.
- **Variables de entorno:** Todos los secretos viven en .env, ignorado por git.

## 9. Seeding de datos

El script **scripts/seed.js** puebla la base de datos. Soporta tres modos:

- **Offline** (por defecto): usa *seeders/heroes.json* con 24 héroes curados.
- **--live**: descarga desde la SuperHero API real.
- **--reset**: recrea las tablas desde cero antes de poblar

## 10. Despliegue

Local con Docker:

```
docker-compose up -d      # levanta Postgres + app
npm run db:reset          # crea tablas y seeds
npm start                 # http://localhost:3000
```

## 11. Lógica de negocio destacada

| Endpoint                 | Rol   | Modifica BD | Descripción                                   |
|--------------------------|-------|-------------|-----------------------------------------------|
| POST /api/heroes         | admin | Si          | Alta de un héroe en el catálogo               |
| PATCH /api/heroes/:id    | admin | Si          | Edición de datos del héroe                    |
| DELETE /api/heroes/:id   | admin | Si          | Baja en cascada del héroe                     |
| POST /api/store/buy      | user  | Si          | Compra atómica (decrementa coins y stock)     |
| POST /api/store/add-hero | user  | Si          | Añade un héroe a la colección del usuario     |
| POST /api/teams          | user  | Si          | Crea un equipo de combate                     |
| PATCH /api/teams/:id     | owner | Si          | Renombra y reasigna héroes (max 6, validados) |

## 12. Retos encontrados y soluciones

| Reto                                              | Solución                                                                  |
|---------------------------------------------------|---------------------------------------------------------------------------|
| ESM + Sequelize CLI (no soporta import nativo)    | Mantener config como .cjs; dotenv cargado al arranque.                    |
| dotenv no cargado en index.js rompía sesiones     | Importar 'dotenv/config' como primera sentencia.                          |
| Inconsistencia import * as X vs export default {} | Servicios exportan funciones nombradas (export async function).           |
| Race condition en compras                         | Transaccion Sequelize con lock: true sobre users y products.              |
| Rate limit / dependencia de la SuperHero API      | Dataset offline (heroes.json); flag --live para importar 731 oficiales.   |
| Filtros/orden seguros sin reinventar la rueda     | Middleware queryFeatures con whitelist de campos y operadores.            |
| Doble flujo de auth (Pug + API)                   | POST /login crea sesion Y devuelve JWT que app.js guarda en localStorage. |

## 13. Conclusiones

**HEROback** consigue un backend completo, modular y documentado que satisface los requisitos académicos. La separación en capas, junto con Sequelize y los middlewares reutilizables, facilita la evolución del proyecto y la incorporación de nuevas fuentes de datos o nuevos consumidores (web, móvil, integraciones).

## 14. Mejoras futuras

- Tests unitarios + integración (vitest / jest + supertest).
- Migraciones Sequelize formales en lugar de sync({ alter: true }).
- Refresh tokens y revocación de JWT.
- Tabla user\_roles separada para soportar múltiples roles por usuario.
- Rate limiting (express-rate-limit) y helmet para cabeceras de seguridad.
- CI/CD con GitHub Actions.